

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

**Chandubhai S. Patel Institute of Technology (CSPIT),
CHARUSAT**

Department of Information Technology / Computer Engineering

ITUE301: Advanced Web Development Frameworks

Open-Book Practical Examination | B.Tech. Semester 5 | AY 2026–27

Date	24 / 08 / 2026
Branch	CE / CSE / IT / AIML
Duration	2 Hours
Total Marks	20 Marks (4 Marks × 5 Tasks)
Tech Stack	React (frontend) + Express.js (backend) + MongoDB with Mongoose
Internet	Permitted for official documentation, MDN, Stack Overflow reference.
AI Tools	Permitted: ChatGPT, GitHub Copilot, Claude, Gemini, etc. may be used. Students must be able to explain every line of submitted code during viva.
Code Sharing	Strictly prohibited: identical submissions across students will be awarded zero.

General Instructions

1. Read the complete question paper before starting implementation. Plan your folder structure first.
2. Attempt all five tasks in your allotted set. Partial implementation earns partial marks: do not leave any task blank.
3. Use React for frontend tasks, Express.js for backend tasks, and MongoDB with Mongoose for database tasks. No other database is permitted.
4. GitHub repository must be public and named exactly: `itue301-exam-[roll-number]-[batch]` (e.g. `itue301-exam-24it045-A`).
5. Use a `.env` file for the MongoDB connection string and PORT. Do not hardcode credentials. Commit a `.env.example`; never commit the real `.env`.
6. Backend must start with `node server.js` or `npm start`. Document this in `README.md`.
7. AI code-generation tools are permitted for this exam (see AI Tools row above). Regardless of how the code was produced, you must be able to explain it in viva: code you cannot explain will be marked zero for that task.
8. Sharing code with another student is strictly prohibited. Copying another student's code results in a zero for both students.

9. Include a README.md with project name, frontend/backend run instructions, and MongoDB setup steps.
10. Final submission must include **both** of the following before the examination window closes: (a) your public GitHub repository link together with the final commit SHA, and (b) the PDF report named as specified in your set. Submit both via the Google Form shared by faculty. Late submissions lose 1 mark per 5 minutes beyond a 5-minute grace period.

SET B: FitZone Gym & Class Booking System

ITUE301 · Open-Book Practical Examination · CSPIT, CHARUSAT · 24/08/2026

Scenario: FitZone gym currently books classes through a WhatsApp group, leading to double-bookings and no-shows. You are hired to build a booking system where members reserve trainer-led classes, trainers see their schedule, and admin manages the roster.

Data Entities:

- **Member:** name, email, phone, membershipType (basic | premium | platinum)
- **Trainer:** name, specialization, available (Boolean)
- **ClassBooking:** memberId (ref), trainerId (ref), className, date, timeSlot, status (booked | attended | cancelled)

User Roles:

Member

Trainer

Admin

Mandatory Requirements

- React frontend with at least three distinct pages (Login, Classes, MyBookings).
- Client-side routing: Classes and MyBookings must be protected routes.
- A reusable TrainerCard component that accepts props and renders trainer details.
- Global auth state via useState / useContext or Zustand.
- Express backend exposing at least 5 REST endpoints at /api/v1/.
- All protected routes pass through a custom authGuard middleware; a global requestLogger is applied to every request.
- MongoDB collections for Member, Trainer, and ClassBooking with Mongoose schemas.
- Schema-level validation (required, enum, min) on all three models.
- Lazy-loaded route for an Admin Panel using React.lazy + Suspense.
- API call from React using fetch or Axios; display a loading state while data is fetched.

Task 1: React Component Architecture

4 Marks

Create the basic React frontend for the FitZone Gym & Class Booking System.

Create the following page/components: LoginPage, ClassesPage, MyBookingsPage, TrainerCard.

Place reusable components inside a suitable /components folder.

The TrainerCard component must accept the following props: name, specialization, available.

Display all three values.

The appearance of the trainer's availability should change according to its value. For example: available = true → "Available", available = false → "Fully Booked": these may use different CSS classes.

Use props to pass trainer data from the parent component to TrainerCard.

Detailed UI styling is not required. Focus on component structure, composition, and correct use of props.

Hint: an object map such as { true: 'Available', false: 'Fully Booked' } keeps the conditional clean.

Task 2: React Routing and State Management

4 Marks

Configure React Router with the following routes:

Route	Component
/	LoginPage
/classes	ClassesPage (protected)
/my-bookings	MyBookingsPage (protected)
/admin	AdminPanel (lazy-loaded)

Create a navigation component containing links to all routes.

Navigation must be performed using React Router links without a full-page reload.

In ClassesPage, create a simple booking form containing at least: trainer, class name, date, time slot.

Use useState to manage the form data.

At least two state values must be used meaningfully. For example: selected trainer, selected time slot.

Display the selected trainer or another chosen value on the page as the state changes.

Implement an AuthContext holding { member, token, role } with login() and logout() functions. A ProtectedRoute wrapper must redirect unauthenticated users to /.

Hint: pass the entire context value as one object: value={{ member, token, login, logout }}.

Task 3: Express REST API + Middleware

4 Marks

Build the Express backend for managing class bookings.

Implement the following REST endpoints:

Method	Endpoint	Purpose
POST	/api/v1/auth/login	Authenticate member, issue token
GET	/api/v1/trainers	Return all trainers (public)
POST	/api/v1/bookings	Create a new class booking (protected)
GET	/api/v1/bookings/my	Return the logged-in member's bookings (protected)
PATCH	/api/v1/bookings/:id/status	Update booking status (protected)

Create a custom requestLogger middleware that logs [METHOD] [PATH] [STATUS] [RESPONSE-TIME ms] for every request, using `res.on('finish')` to capture the final status code. Apply it globally.

Create an authGuard middleware that validates the Bearer token and attaches `req.member`. Apply it to every route except `/auth/login` and the public trainer listing.

Implement a global error-handling middleware as the last middleware, returning structured JSON instead of a raw error stack.

Use appropriate HTTP status codes: 200 for successful GET, 201 for successful POST, 400 for a validation error, 401 for missing/invalid auth, 500 for an unhandled server error.

Test the APIs using Postman or Thunder Client.

Hint: `res.on('finish', () => { ... })` lets the logger capture the response after it has actually been sent.

Task 4: REST API Consumption in React

4 Marks

Use the Express API developed in Task 3 from the React frontend.

In `ClassesPage`, retrieve trainer information using: `GET /api/v1/trainers`.

Use `fetch` or `Axios`.

Use `useEffect()` so that the API request is made when the component mounts.

Maintain three states: `trainers`, `loading`, and `error`.

The page must: 1) Display a loading message or indicator while the request is in progress. 2) Display an error message if the request fails. 3) Display the trainer data, rendered via `TrainerCard`, after a successful request.

Display at least: trainer name, specialization, and availability.

The trainer information must be rendered from the API response and must not be hardcoded in the React component.

Add a client-side search input that filters the already-fetched trainers array by specialization, without triggering a new API request.

Hint: keep the full array in one state; derive the filtered list with `.filter()` at render time.

Task 5: MongoDB + Mongoose Schema Design and Validation**4 Marks**

Create the Mongoose-based implementation for the FitZone database. Connect to MongoDB using Mongoose.

Create the following three schemas.

Schema	Field Requirement
Member	name required; email required, unique; membershipType enum, default 'basic'
Trainer	name required; specialization required; available Boolean, default true
ClassBooking	memberId ref Member; trainerId ref Trainer; date required; timeSlot required; status enum, default 'booked'

The membershipType field must use: basic, premium, platinum.

The ClassBooking status field must use: booked, attended, cancelled.

Use Mongoose references for: memberId → Member, trainerId → Trainer.

POST /api/v1/bookings must validate the request body, save the booking to MongoDB, and return 201 on success.

GET /api/v1/bookings/my must use .populate('memberId', 'name email') and .populate('trainerId', 'name specialization').

Connect MongoDB using a connection string stored in .env. For example:

MONGO_URI=your_mongodb_connection_string.

Demonstrate at least one validation failure, such as a missing required field or an invalid membershipType, and return a meaningful JSON error response instead of exposing the raw Mongoose error object.

Hint: catch err.name === 'ValidationError' and map Object.values(err.errors) to plain messages before sending the response.

Submission Requirements

Public GitHub repo itue301-exam-[roll-number]-[batch] with /frontend, /backend, README, .env.example. PDF [RollNo]_SetB_Report.pdf with screenshots: (1) ClassesPage with trainer cards, (2) Postman showing 201 on booking creation, (3) MongoDB showing a saved document.

Final Submission (both required): ① GitHub repository link + final commit SHA · ②

[RollNo]_SetA_Report.pdf: Upload pdf to Google Classroom / Microsoft Team Assignment before the exam ends.